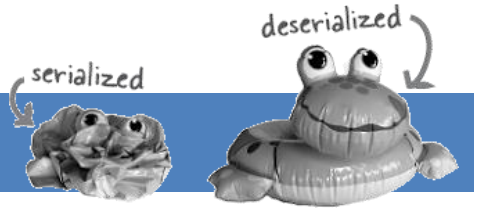
GROEIPROJECT
MODULE 6: PERSISTENTIE



1. VOORBEREIDING

1.1. Open het groeiproject en maak daarin een zesde module: `m6_persistentie`

1.2. Kopieer van module `m1_herhaling` de hele **`be.`** Package (met o.a. de oorspronkelijke basis- en multiklasse en ook de data-klasse).

1.3. Pas de `build.gradle` van deze module aan: er is deze keer GEEN `application`, dus ook geen `mainClassName`, maar je moet wel een dependency toevoegen om **junit** en **hsqldb** te kunnen draaien:

```
dependencies {
    testImplementation platform('org.junit:junit-bom:5.10.0')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    implementation 'org.hsqldb:hsqldb:2.7.2'
}
```

1.4. Maak in de module `m6_persistentie` een nieuwe directory (dus **NAAST** `src`) met de naam `db`. Hierin komen de databestanden te staan.



← Hiernaast een screenshot van de projectstructuur na deze voorbereidende stappen.

2. OBJECT SERIALIZATION

2.1. Zowel in de basisklasse als in de multiklasse implementeer je de `serializable` – interface.
Voorzie ook telkens een static attribuut `serialVersionUID` en zet daarboven de annotation `@Serial`
Kies in de basisklasse een drietal minder belangrijke attributen en markeer die als `transient`.

2.2. Maak een nieuwe subpackage **`persist`** en daarin een klasse `XxxSerializer` (in ons geval: `DictatorsSerializer`)

a. Voorzie een final attribuut `FILENAME` dat via de constructor wordt ingesteld.

b. voorzie een methode `serialize` met volgende signatuur (aan te passen naar jouw situatie):

```
public void serialize(Dictators dictators) throws IOException
```

Serialiseer het multiobject dat als parameter binnenkomt naar een databestand in de directory `db`. Gebruik een relatief pad (in ons voorbeeld: `"db/dictators.ser"`) Een eventuele IOException wordt verder gethrowed.

c. voorzie een methode `deserialize` met volgende signatuur (uiteraard aan te passen naar jouw situatie):

```
public Dictators deserialize() throws IOException,
                                      ClassNotFoundException
```

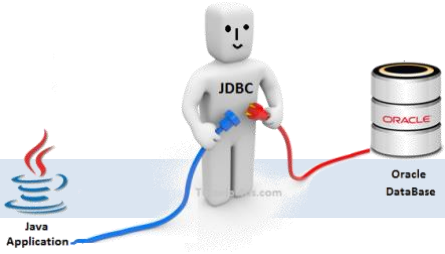
Deserialiseer vanuit het databestand in een **nieuw multiobject** en retourneer het. Een eventuele IOException of ClassNotFoundException wordt verder gethrowed.

2.3. Maak onder de test-map een testklasse `XxxSerializerTest` (in ons geval dus `DictatorsSerializerTest`)
(zie screenshot van projectstructuur op het einde van dit document.)

a. Voorzie een `@BeforeEach` methode die de `XxxSerializer` aanmaakt en het multiobject vult vanuit de dataklasse.
b. Schrijf een testmethode `testSerialize` die controleert of er geen IOException optreedt bij het serialiseren
c. Schrijf een testmethode `testDeserialize` die via assertEquals controleert of het gedeserialiseerd object dezelfde inhoud bevat als het oorspronkelijk multiobject en er geen exception optreedt.
Belangrijk: Zorg er eerst voor dat het multiobject een `equals` en een `hashCode` methode heeft!

Run beide tests vanuit Gradle en controleer of er onder de directory **db** effectief een bestand ***.ser** aangemaakt werd.

3. JAVA DATABASE CONNECTION (JDBC)



TIP: Baseer je bij de uitwerking van de volgende opdrachten op de code-voorbeelden die je op Canvas vindt.

3.1. Omdat we met een relationele databank gaan werken, moeten alle records voorzien zijn van een **primary key** (PK). De databaseomgeving die we hier gebruiken (**hsqldb**) doet de nummering van de PK zelf. Vandaar enkele aanpassingen in de **basisklasse** (in ons geval `Dictator`):

a. Maak een extra attribuut `id` van het type `int`. Dit wordt de PK.
b. Maak een kopie van de bestaande constructor en voeg een extra `id` parameter toe
c. In de oude constructor, roep je de nieuwe constructor aan met de `id` by default op `-1`
d. Voorzie een getter en setter voor `id`

3.2. In de package `persist` maak je een **interface** `XxxDao` (in het voorbeeld is dit `DictatorDao`) met volgende abstracte methoden (pas uiteraard de parameters aan naar jouw context):

```
boolean insert(Dictator dictator);
boolean delete(String naam);
boolean update(Dictator dictator);
Dictator retrieve(String naam);
List<Dictator> sortedOn(String query);
```

3.3. Implementeer deze interface in een nieuwe klasse `XxxDbDao` (in ons voorbeeld is dit `DictatorDbDao`). Werk deze klasse verder uit zoals hieronder wordt beschreven.

3.4. Maak in de klasse `XxxDbDao` een verbinding met de database:

a. Via de **constructor** komt het databasePath binnen. Maak een `Connection` met de database. We gebruiken eenzelfde Connection voor alle database interactie in de DAO; dus voorzie deze Connection als **private attribuut** zodat ze in elke methode bereikbaar is.

b. In alle methoden die je in deze klasse uitwerkt moeten alle `SQLExceptions` afgehandeld worden door ze te loggen met als level SEVERE en voorzien van een *eigen* specifieke foutboodschap, aangevuld met de *oorspronkelijke* SQLException-message. Voorzie dus ook een private **Logger** om dit te doen.

c. Voorzie een methode **`close`** die de `connection` sluit (als die niet null is).

d. Voorzie een private methode **`createTable`** die de database-tabel en alle velden definieert en één veld aanduidt als PK. In ons voorbeeld ziet dat er als volgt uit:

```
"CREATE TABLE dictatortable (id INTEGER IDENTITY", name VARCHAR(30), gender
VARCHAR(5), ... "
```

Roep `createTable` aan vanuit de constructor nadat de connectie gemaakt is.

e. Voeg in `createTable` voor de create een statement toe dat de tabel dropt als die reeds bestaat. Voor het Dictatorvoobeeld is de SQL instructie: **`"DROP TABLE dictatortable IF EXISTS"`**

3.5. In de klasse `XxxDBDao` implementeer je vervolgens de **CRUD**- methoden:

a. **`insert:`** Als parameter komt een basisobject binnen (in ons geval: `Dictator`), dat moet toegevoegd worden aan de database (`"INSERT INTO…"`). Maak hier gebruik van een

`PreparedStatement`, met plaatshouders voor elke attribuutwaarde van de basisklasse.
Opgelet:
- de PK wordt door hsqldb zelf aangemaakt, dus voor dat veld geef je als waarde `NULL` mee.
- een `LocalDate`-waarde moet je omzetten naar een `java.sql.Date` : zie slides!
- een enumtype moet je omzetten naar String : zie slides!
- vergeet niet om elk (`Prepared`)`Statement` weer te sluiten (de connection niet; die blijft open)

b. **`retrieve`**: Als parameter komt een **String** binnen waarop gezocht moet worden (`"SELECT * FROM… WHERE…"`). Als return-value wordt een gevuld basisobject teruggegeven.
Opgelet:
- een `Date`-waarde moet je omzetten naar een `localDate`: zie slides!
- een String moet je omzetten naar een enumtype: zie slides!
- Deze keer lees je ook de PK in en gebruik je de nieuwe constructor die je in 3.1 gemaakt hebt.

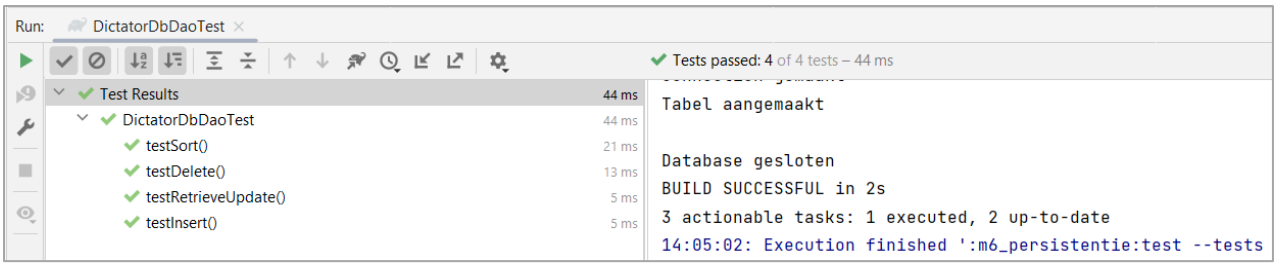
c. **`update`**: Als parameter komt het gewijzigde basisobject binnen; als return-value geef je een boolean terug. Zoek het record ahv de PK en wijzig de andere veldwaarden (`"UPDATE… SET… WHERE…"`). Gebruik hier een `PreparedStatement`.

d. **`delete`**: Als parameter komt een **String** binnen waarop gezocht moet worden. Verwijder het record dat hieraan voldoet.
Opgelet: Als er als parameter een **"*"** meegegeven wordt, dan verwijder je **ALLE** records.

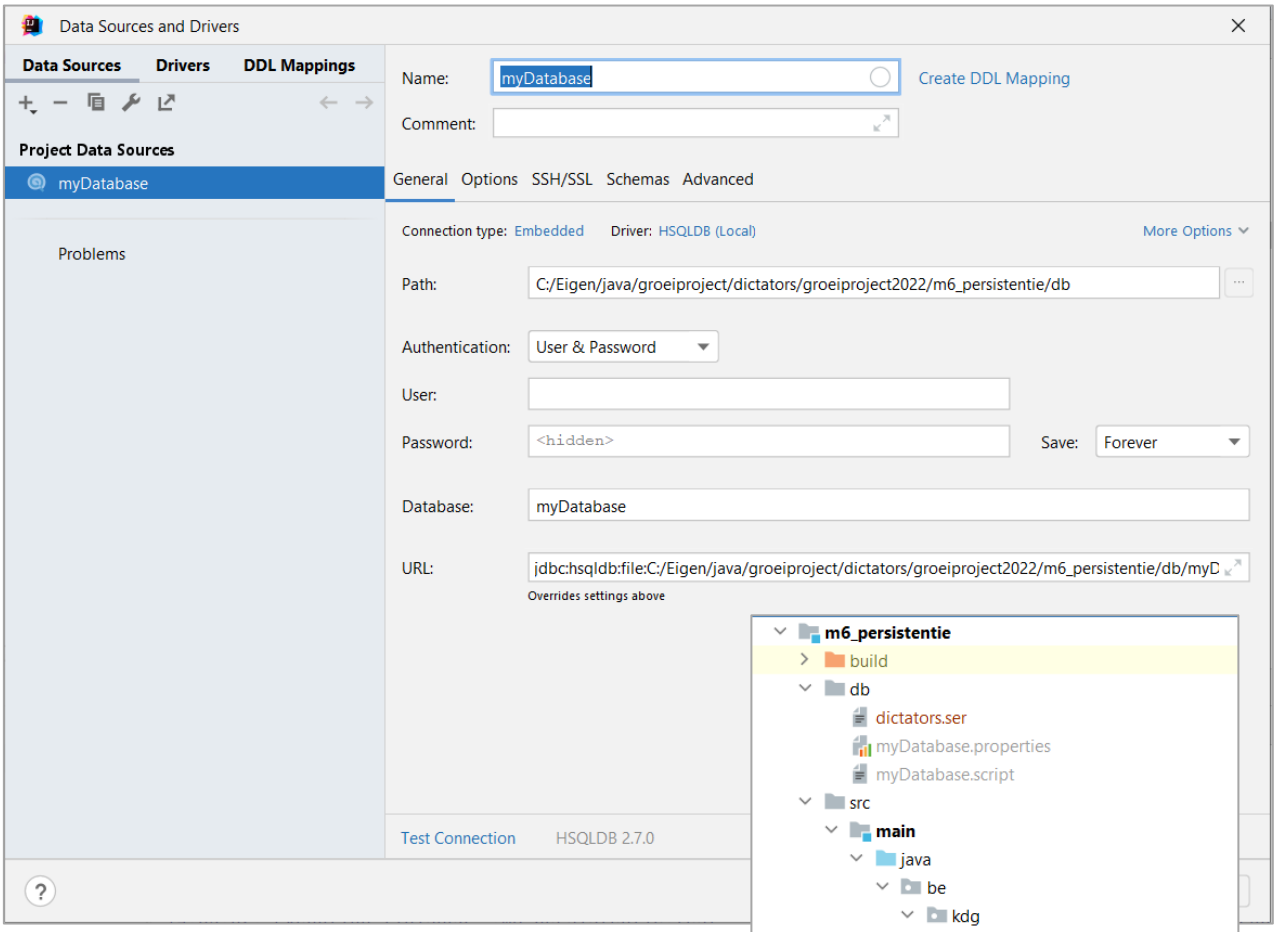
e. **`sortedOn`**: Als parameter komt een query-string binnen (bijvoorbeeld: "`SELECT * FROM dictatortable ORDER BY name`"). Laat het sorteren over aan de databank (`ORDER BY`), dus geen gebruik maken van de multiklasse! Als resultaat geef je een gesorteerde List terug.
Schrijf nu een aantal andere `sortedOnXXX` methoden (bijvoorbeeld `sortedOnName`, `sortedOnBirth`…) die de bovenstaande `sortedOn` methode oproepen met een specifieke query als parameter.

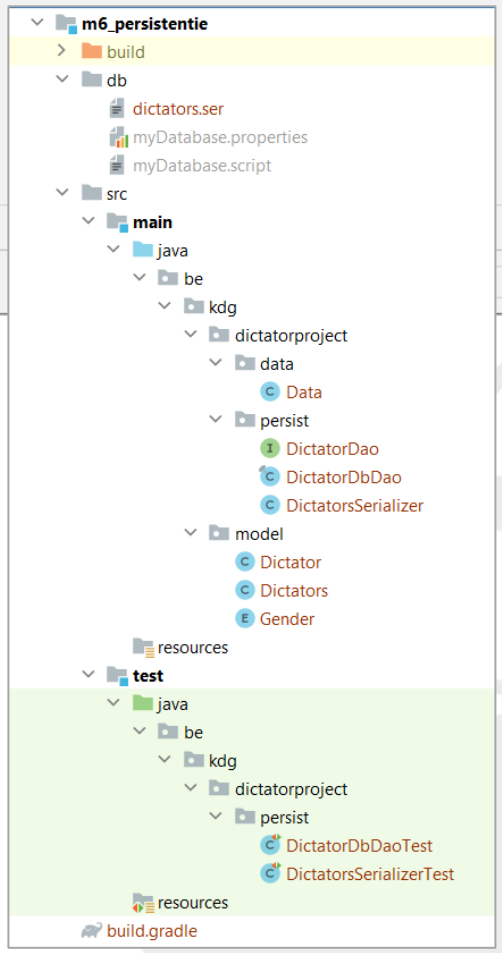
3.6. Schrijf een testklasse `XxxDbDaoTest` (in ons geval: `DictatorDbDaoTest`); daarin testen we alles uit:
   a. In een @`BeforeAll` methode instantieer je éénmalig de Dao-klasse.
   b. In een @`AfterAll` methode roep je de close methode op.
   c. In een @`BeforeEach` methode vul je de databank vanuit je dataklasse
   d. In een @`AfterEach` methode verwijder je alle records uit de databank (dus delete("*"))
   e. `testInsert`: controleer of er evenveel records in de databank zitten als elementen in de Data-klasse
   f. `testRetrieveUpdate`: doe een retrieve van een bepaald record en verander vervolgens de naam. Doe update en retrieve opnieuw. Vergelijk met assertEquals.
   g. `testDelete`: Verwijder een bepaald record; controleer op het aantal elementen in de databank; en probeer het nogmaals te verwijderen (dat zou moeten mislukken)
   h. `testSort`: Vergelijk een gesorteerde List uit de Data-klasse met het resultaat van de `sortedOnXxx` methode van je Dao-klasse.

Run alle tests via gradle en kijk of alle tests goen kleuren (zie volgende pagina)



3.7. Bekijk de records van de databank ook eens via het "*Database view*" venster (rechts). Daar moet je dan wel eerst de datasource instellen:





3.8. Een screenshot van de projectstructuur op het einde van je opdracht vind je hier rechts →